Code Narrative

An implementation guide to the agent-based model of
Driving in the wrong direction? Modelling policy mixes for EV adoption

Miquel Bassart i Loré[1,2] and Daniel Torren-Peraire[3,4,5]

[1]Universität Bielefeld
[2]Université Paris 1 Panthéon-Sorbonne
[3]Universitat Autònoma de Barcelona
[4]Università Ca' Foscari Venezia
[5]International Institute for Applied Systems Analysis

Abstract

This document is a guided tour of the code that implements the agent-based model of the paper. It is organised to follow the model, one time step from beginning to end, then each submodule in the order the paper introduces it and states throughout which function implements which equation. Section 13 gives a symbol-to-variable map, and Section 11 documents mechanisms that exist in the code but are switched off in every configuration used for the paper.

---

1 Orientation

1.1 Layout

Directory	Contents
`package/model/`	The model itself: agents, landscapes, orchestration
`package/resources/`	Run harness (`generate_data`, `load_in_controller`) and I/O helpers
`package/constants/`	`base_params_*.json` experiment configurations, `vary_*.json` sweep definitions
`package/analysis/`	The paper's policy experiments (Section 4)
`package/generating_data/`	Calibration, sensitivity and robustness runners
`package/plotting_data/`	Figure scripts paired with the runners
`package/calibration/`	Empirical data assembly and simulation-based-inference calibration
`package/calibration_data/`	California input series

1.2 Running the model

Dependencies are managed with `uv` (Python 3.13+); `uvsync` creates the environment. Scripts resolve configuration paths relative to the repository root and use absolute imports, so they are invoked as modules from the root:

```
uv run python -m package.generating_data.calibration_gen
```

Each `*_gen.py` script writes a timestamped directory under `results/` and most call their matching `*_plot.py` immediately afterwards. The repository `README.md` carries the full map from paper figure to script and configuration file.

1.3 The single-run entry point

Every experiment ultimately calls `generate_data(parameters)` in `package/resources/run.py`. It loads the pickled California input series, derives the total horizon, constructs a `Controller`, and steps it to the end:

```
parameters["time_steps_max"] = (parameters["duration_burn_in"]
                                + parameters["duration_calibration"]
                                + parameters["duration_future"])
controller = Controller(parameters)
while controller.t_controller < parameters["time_steps_max"]:
    controller.next_step()
return controller
```

The returned `Controller` *is* the result: all recorded time series hang off it (Section 10), and experiment scripts pickle it or extract the series they need. `run.py` also holds a family of `*_parallel_run` wrappers that map `generate_data` over a list of parameter dictionaries with `joblib`, each returning a different projection of the results; these are what the experiment scripts actually call.

2 One time step

`Controller.next_step()` is short, and its order is the model's update sequence:

```
def next_step(self):
    self.update_time_series_data()
    self.cars_on_sale_all_firms = self.update_firms()
    self.second_hand_cars = self.get_second_hand_cars()
    self.consider_ev_vec, self.new_bought_vehicles = self.update_social_network()
    self.manage_saves()
    self.t_controller += 1
```

This maps one-to-one onto the numbered sequence in the paper's Figure 1:

Step	Paper	Code
0	Exogenous variables and policies update	`update_time_series_data()`
1	Firms receive user and competitor information	`FirmManager.next_step()` receives `consider_ev_vec` and `new_bought_vehicles` from the previous step
2	Firms update product mix and prices	`Firm.choose_cars_segments()`, fired with probability θ
3	Firms innovate	`Firm.innovate()`, fired with probability ι
4	Used market scraps and reprices	`SecondHandMerchant.next_step()`
5	Users choose to keep, buy new, or buy used	`SocialNetworkUsers.update_VehicleUsers()`
6	Emissions are calculated	`update_emisisons()`, within the user update
7	Social parameters update for the next step	`calculate_ev_adoption()`

Two ordering details matter. First, firms act on information from the previous period. `consider_ev_vec` and `new_bought_vehicles` are returned by the user update at the *end* of a step and consumed by the firm update at the *start* of the next, which makes the market-research channel a lag rather than an instantaneous feedback. Second, EV consideration is recomputed after all purchases are settled, so a user's threshold is evaluated against the network state their own purchase has already contributed to.

2.1 Exogenous inputs and policy application

`update_time_series_data()` indexes precomputed vectors at `t_controller` rather than recomputing paths per step. It sets the carbon price, the effective gasoline cost, the grid emissions intensity, the electricity price net of subsidy and gross of carbon price, the new and used EV rebates, and the production subsidy. It also flips the EV research and production switches when their start times arrive:

```
if self.t_controller == self.ev_research_start_time:
    for firm in self.firm_manager.firms_list:
        firm.ev_research_bool = True
        firm.list_technology_memory = (firm.list_technology_memory_ICE
                                       + firm.list_technology_memory_EV)
```

The electricity price shows the two policy channels composing:

```
self.electricity_price = (self.electricity_price_vec[t] * (1 - subsidy_prop)
                          + self.carbon_price * self.electricity_emissions_intensity)
```

The subsidy scales the pre-tax price; the carbon price is added on top of the subsidised price, weighted by grid carbon intensity. Because intensity falls over the policy period under the assumed linear decarbonisation, the carbon price component of electricity shrinks as the grid cleans up, while the gasoline component does not.

Policy values arrive as time series built by `gen_time_series_calibration_scenarios_policies()` from the `parameters_policies` block: `States` switches each instrument on or off, `Values` sets its intensity. Instruments are applied from January 2024 at full intensity and held constant, so the series are step functions.

3 Object model

`Controller` owns the whole simulation and is the only object that knows the calendar. It constructs and holds:

Object	Responsibility
`SocialNetworkUsers`	The whole user population as vectors, plus the network and the choice machinery
`FirmManager`	The firm population, market segmentation, and the competition statistics $W_{s,t}$
`Firm` $\times J$	One manufacturer: technology memory, product mix, pricing, innovation
`NKModel_ICE`, `NKModel_EV`	The two technology landscapes
`SecondHandMerchant`	The consolidated used car dealer and its stock

Note that users are *not* individual objects in the hot path. `SocialNetworkUsers` holds population attributes as NumPy vectors (`beta_vec`, `gamma_vec`, `chi_vec`, `d_vec`) and evaluates utility as a users-by-vehicles matrix; `VehicleUser` is a thin record used for bookkeeping. Cars come in two kinds, and the distinction is load-bearing: `CarModel` is a *design* on sale (an attribute vector and a price, not a physical car), while `PersonalCar` is an *instance* that exists only once a design is bought, since production is on demand. Both carry `transportType`, which is `2` for ICEVs and `3` for EVs, a convention worth memorising, as it appears in masks throughout the code.

4 Car users

4.1 Choice set construction (Eq. 1)

Two mechanisms restrict what a user may choose. Activation: with probability η (`prob_switch_car`) a user considers replacing their car at all; otherwise their only option is what they already own. EV consideration: users whose imitation threshold is not met cannot see EVs.

The second is implemented as a mask rather than by building per-user lists. `gen_mask()` constructs a users-by-vehicles boolean matrix by outer product, and `masking_options()` sets masked entries to $-\infty$ before exponentiation:

```
not_ev_vec = np.array([v.transportType == 2 for v in vehicles], dtype=bool)
consider_ev_all_vehicles = np.outer(consider_ev_vec == 1, ones)
dont_consider_evs_on_ev = np.outer(consider_ev_vec == 0, not_ev_vec)
ev_mask_matrix = (consider_ev_all_vehicles | dont_consider_evs_on_ev).astype(int)
```

A user who considers EVs gets every vehicle; a user who does not gets only ICEVs. This is Eq. 1's case split, vectorised.

4.2 Utility (Eq. 2–4)

`vectorised_calculate_utility_new_cars()` and its second-hand counterpart evaluate Eq. 2 as a matrix. The four terms are willingness to pay for quality with diminishing returns α, willingness to pay for range with diminishing returns ζ (range being battery or tank size times efficiency, depreciated by age), lifecycle emissions priced at γ_i, and lifecycle costs.

The discounted-sum terms of Eq. 3 and Eq. 4 share a single helper, `_lifecycle_cost_term()`, which implements the closed-form geometric series of the naive-expectations case: a user assumes today's energy price and emissions intensity persist indefinitely, so the infinite discounted sum collapses to

$$D_i \frac{(1+r)(1-\delta_{a,t})\, c_{a,t}}{\omega_{a,t}\,(r-\delta_{a,t}-r\delta_{a,t})}.$$

The purchase-price term is present only when the evaluated car is not the user's own, and it is net of the expected sale price of the car being replaced.

Because a user's own car is always in the choice set, its utility is computed by a separate path (`generate_utilities_current()`) with a small cache (`_build_cv_cache()`, `_update_cv_cache_row()`) so that owned-car utility is not rebuilt from scratch each step.

4.3 Logit choice (Eq. 5)

Choice is sampled from the masked, exponentiated utilities. Users are activated sequentially in random order, because a used car can only be bought once and the stock must be updated between choices. `masking_options()` subtracts the row maximum before exponentiating for numerical stability, and `user_chooses()` samples from the unnormalised cumulative distribution:

```
cumsum = np.cumsum(individual_specific_util_kappa)
choice_index = int(np.searchsorted(cumsum, u_draw * cumsum[-1], side='right'))
```

Scaling the uniform draw by `cumsum[-1]` avoids normalising the row, which is where the κ-weighted denominator of Eq. 5 would otherwise be formed. If a user has no admissible option, which happens only in the first steps, they keep their current car.

The consequences of a choice are handled in the same function: the replaced car is transferred to the used car dealer at its purchase price, unless it is an initialisation car or already at scrap value, in which case it leaves the simulation.

4.4 Social imitation (Eq. 6–7)

`create_network()` builds a Watts–Strogatz small-world graph from `SW_network_density` and `SW_prob_rewire`, stored as a normalised adjacency matrix. `calculate_ev_adoption()` computes each user's neighbourhood EV share and compares it against `chi_vec`, returning the binary `consider_ev_vec` that gates the next step's choice sets.

Thresholds are drawn from a beta distribution in `gen_chi()`, with a fixed `proportion_zero_target` (0.5%) of the population pinned at $\chi_i = 0$ so that contagion has a seed. Note that the threshold is a gate on *consideration*, not on choice: a user above their threshold still has to prefer an EV on utility grounds, which is the behaviour–attitude gap the paper discusses.

5 Car supply

5.1 Segmentation (Appendix A.2.1)

`FirmManager` partitions the population into segments on willingness to pay for quality (`num_beta_segments`, $R^\beta = 8$) and for emissions reduction (`num_gamma_segments`, $R^\gamma = 2$), crossed with the binary EV consideration state. Each segment is represented by preference parameters at the corresponding quantile, giving the representative consumer profiles that firms price against.

5.2 Competition state (Eq. 9)

`update_W_immediate()` computes the current κ-weighted sum of lifetime utilities on offer per segment, and `update_market_data_moving_average()` maintains the twelve-month moving average that becomes $W_{s,t}$. Crucially, for segments that do not consider EVs only ICE cars enter the sum, matching Eq. 9's case split. The result is passed down to firms as `W_vec` together with `I_s_t_vec`, the segment populations.

5.3 Optimal pricing (Eq. 10)

`Firm.calc_optimal_price_cars()` implements Eq. 10 directly, including its Lambert $\mathbb{W}$ term, as a cars-by-segments matrix operation:

```
U = term1 + term2 + term3 + term4          # utility at P = marginal cost
exp_input = (self.kappa * U - 1) - np.log(self.W_vec[np.newaxis, :])
LW = lambertw(np.exp(exp_input), 0).real
P = C_m_cost[:, np.newaxis] + (1.0 + LW) / self.kappa
```

Two subsidies enter here, and they enter differently. This is exactly the distributional distinction the paper draws between a production subsidy and a purchase rebate:

```
C_m_cost[ev_mask]  = np.maximum(0, C_m[ev_mask] - self.production_subsidy)
C_m_price[ev_mask] = np.maximum(0, C_m[ev_mask]
                                - (self.production_subsidy + self.rebate
                                   + self.rebate_calibration))
```

`C_m_cost` is the cost the firm actually bears, and it sets the price floor. `C_m_price` is the cost relevant to the consumer's perceived price, and it enters the utility term. The production subsidy appears in both, so the firm captures part of it as rent; the rebate appears only in the second, so it passes through to the buyer. Both are floored at zero, which is the zero-lower-bound the paper describes and the mechanism behind the finding that stacking a rebate on a production subsidy only inflates profits once the bound binds.

5.4 Product mix (Appendix A.2.2)

`choose_cars_segments()` implements the sequential heuristic: build the segments-by-technologies expected-profit matrix (`set_utility_and_profit_grid_production()`, `calc_predicted_profit_segments_production()`), repeatedly take the most profitable cell, add that technology at that segment's optimal price, delete the segment row, lock the price for that technology across remaining segments, and recompute, repeating until `max_cars_prod` (Y^+) is reached or the matrix empties. Locking the price is what allows a design to serve several segments at one price while being less competitive in the poorer ones.

The whole procedure fires only with probability θ (`prob_change_production`), representing price rigidity and information delay.

5.5 Innovation (Eq. 11–15)

`innovate()` fires with probability ι (`prob_innovate`), which is how failed innovation is represented. It builds the candidate set of Eq. 11 from unexplored technologies at Hamming distance one from the current search location on each landscape, plus the current locations themselves so that not searching remains an option (`generate_neighbouring_technologies()`, `gen_neighbour_strings()`). Candidates are scored by expected profit at their best segment (`calc_predicted_profit_segments_research()`) and one is drawn by a logit with research intensity λ (`select_car_lambda_research()`), balancing exploration against exploitation.

Memory follows Eq. 14–15. `update_memory_timer()` increments a per-technology idle counter, zeroed for anything researched or in production, and `update_memory_len()` evicts the longest-idle technology not in production once `memory_cap` (M^+) is exceeded.

5.6 NK landscapes (Supplementary 3.1)

`nkModel_ICE.py` and `nkModel_EV.py` are near-identical, differing in dimensionality: both map an N-bit string with epistasis K to attributes, but the EV landscape carries battery size as an additional attribute, so ICE has three attributes and EV four. `generate_fitness_landscape()` builds the contribution tables, and `calculate_fitness_vectorized()` evaluates attributes for many strings at once. Attribute ranges and the cost-attribute correlations ρ come from the configuration.

Landscapes are constructed once per run by `setup_ICE_landscape()`/`setup_EV_landscape()` and shared across firms, so firms differ only in where they have searched, which is the source of emergent specialisation. `find_min_fitness_string()` provides the initialisation point: firms start one Hamming step from the worst technology evaluated at the median segment.

6 Used car market (Eq. 16–21)

`SecondHandMerchant` is the single consolidated dealer. Its pricing heuristic (`calc_car_price_heuristic()`) implements Eq. 18–20: normalise quality, efficiency and battery size by the maxima over new cars, find each used car's nearest new design in that space, take its price net of rebates, and depreciate by age at the type-specific monthly rate `delta_P` (ξ):

```
closest_prices = np.maximum(first_hand_prices[closest_idxs]
                            - (self.rebate_calibration + self.rebate), 0)
adjusted_prices = closest_prices * (1 - second_hand_delta_P) ** second_hand_ages
```

Because EVs depreciate faster than ICEVs (ξ of 0.87% against 1.16% monthly), the used EV stock holds value differently, which is what makes the used EV rebate weak early in the transition: there is very little used EV stock to subsidise.

`update_stock_contents()` applies the markup μ to form purchase prices (Eq. 16), scraps cars priced below scrap value F (Eq. 21), and enforces the two computational bounds: `max_n`

`um_cars_prop` (V^+, the stock ceiling, 10% of population) and `age_limit_second_hand` (T^+, months unsold before scrapping). The dealer also refreshes owned-car fuel costs from the current gas price each step, which is why a driving-cost shock reaches second-hand cars too.

7 Run structure and controller reuse

A run has three phases, measured in months and set by configuration:

Phase	Length	Purpose
`duration_burn_in`	180	ICEV-only market reaches a steady state; EV research and production are disabled
`duration_calibration`	276	2001–2023, matched against California data
`duration_future`	144	2024–2035 policy period, extended to 2050 for the stability analysis

`manage_burn_in()`, `manage_calibration()` and `manage_scenario()` switch the exogenous regime between phases. Burn-in uses `next_step_burn_in` variants on the firm side, which update product mix without the EV machinery.

The important performance property is that the first two phases are computed once and reused. `setup_continued_run_future()` takes an already-calibrated controller, applies a new future-period policy configuration, and `load_in_controller()` continues stepping from where calibration ended:

```
def load_in_controller(controller_load, base_params_future):
    base_params_future["time_steps_max"] = (controller_load.duration_burn_in
                                            + controller_load.duration_calibration
                                            + base_params_future["duration_future"])
    controller_load.setup_continued_run_future(base_params_future)
    while controller_load.t_controller < base_params_future["time_steps_max"]:
        controller_load.next_step()
    return controller_load
```

Policy scenarios are therefore deep copies of one calibrated controller, which is what makes a Bayesian optimisation over policy intensity affordable: 456 months of burn-in and calibration are not re-simulated per candidate intensity. It also means every policy scenario shares an identical history up to December 2023, so differences between scenarios are attributable to policy rather than to stochastic divergence before it.

8 Policy instruments

The five instruments of the paper enter at different points, summarised here because the entry point determines the economics:

Instrument	Where it acts
Carbon price	Added to gasoline and electricity cost per kWh, weighted by emissions intensity, in `update_time_series_data()`. Affects all users every month.
Electricity subsidy	Scales the pre-tax electricity price. Affects EV users every month.
New EV rebate	Reduces the consumer-facing cost term `C_m_price` in pricing; floored at zero. One-off at purchase.
Used EV rebate	Reduces used EV prices in the dealer's heuristic and in second-hand utility. One-off at purchase.
Production subsidy	Reduces both `C_m_cost` and `C_m_price`, so firms retain part of it. One-off at purchase.

Net public cost is accumulated as *policy distortion* on each actor and aggregated by `calc_total_policy_distortion()` and `calc_net_policy_distortion()`, the latter negated so that a positive value is a cost to the public purse. Carbon price revenue and subsidy outlays therefore net against each other, which is how the near-budget-neutral carbon-price-plus- electricity-subsidy combination arises.

Note that the calibration-period rebates, the simplified federal and state EV incentives of 2010–2023, are carried by separate `rebate_calibration`/`used_rebate_calibration` series so that they can be switched off independently of the policy-period instruments, as the paper's experimental design requires.

9 Calibration

Calibration proceeds in the layers the paper describes.

`calibration_data_inputs.py` assembles the California series (vehicle population, EV sales, gasoline and electricity prices, grid emissions intensity, CPI) into the pickled object `generate_data` loads; `calibration_data_outputs.py` formats the observed targets. `fit_distance.py` fits the monthly distance-driven distribution.

Analytically determined parameters are computed inside the controller rather than configured. `calc_beta_median()` solves for the median willingness to pay for quality such that the optimal price of an average ICEV targeted at an average consumer equals the observed average price; `generate_beta_values_quintiles()` then distributes it across the population by the Californian income distribution, so richer agents get higher β_i. The remaining population draws are `gen_distance()`, `gen_chi()`, `gen_nu()` and `gen_gamma()`.

The innovativeness threshold distribution is calibrated by simulation-based inference in `NN_multi_round_calibration_multi_gen.py`, which uses `sbi` to fit the beta distribution parameters `a_chi` and `b_chi` against 2010–2023 EV uptake over multiple rounds. Consumer intensity of choice κ and the efficiency depreciation rate δ are set by grid search against the target ranges for price, market concentration and car age.

10 Outputs

Recording is opt-in and thinned. `save_timeseries_data_state` enables it and `compression_factor_state` sets the stride, so long multi-seed runs can record sparsely or not at all. Each object has a `set_up_time_series_*`/`save_timeseries_*` pair, and `Controller.manage_saves()` drives them.

Series live on the object that owns the quantity: adoption, prices, emissions, utility and car age on `social_network`; concentration, profit and margins on `firm_manager`. The scalar summaries used as optimisation targets and table entries are computed on demand (`calc_EV_prop()`, `calc_mean_car_age()`, `calc_last_step_HHI()`, `calc_price_range_ice()`), and the aggregation helpers in `run.py` select which of these to return across a parameter sweep.

11 Extensions present in the code but not used in the paper

The model contains three optional mechanisms that are disabled by default and switched off in every configuration in this repository. They produce no result reported in the paper. Each is guarded so that omitting its parameter reproduces the published behaviour exactly; they are retained as a foundation for follow-up work on command-and-control policy and on policy anticipation.

11.1 ICE bans

The paper restricts itself to market-based instruments and explicitly excludes command-and-control policy. Three independent ban levers are nevertheless implemented, each expressed as months after the end of burn-in, the same convention as `ev_production_start_time`, and each defaulting to `None`, meaning never:

Parameter	Effect
`ICE_research_ban_time`	Firms may no longer research or improve ICE technology. `innovate()` stops generating ICE candidates. Existing ICE designs stay in memory and remain sellable.
`ICE_sales_ban_time`	Firms may no longer offer new ICEVs. `choose_cars_segments()` drops ICE from the candidate pool, and `next_step()` immediately flushes any ICE design left on sale rather than waiting for the next θ roll. Already-sold ICEVs remain drivable and resellable.
`ICE_driving_ban_time`, `ICE_driving_ban_penalty`	A per-unit penalty added to effective ICE fuel cost from the ban date onward, folded into `gas_cost_effective_vec`. Reaches second-hand ICEVs, since the dealer refreshes fuel costs from the same series.

The three are additive, so research-only, research-plus-sales, and research-plus-sales-plus-driving scenarios can all be configured. The design choice worth noting is that the driving ban is a cost shock rather than a prohibition: it therefore flows through the existing utility-driven choice mechanism and requires no new decision rule on either side of the market, whereas the sales and research bans are hard constraints on firms' choice sets. All three validate that EV research or production has already begun before the ban binds. The active flags are recomputed every step from the configured times rather than latched by a one-shot equality test, so they survive `setup_continued_run_future()` re-deriving them for a new scenario.

11.2 Forward-looking expectations

`forward_looking_expectations` (default `False`) switches agents away from the naive expectations of the paper, under which the current energy price and emissions intensity are assumed to persist and the discounted sum collapses to the closed form of Section 2. With the flag on, agents instead discount the actual known future path.

`compute_discounted_indices()` precomputes present-value indices by backward recursion, $\text{Index}(t) = z_t + q\,\text{Index}(t+1)$, once per run rather than per agent, so the switch costs nothing per utility evaluation. Unrolling the recursion with a constant z_t reproduces the naive closed form exactly, so it is a strict generalisation. With the flag off the indices are still computed but never read by the utility formulas.

The boundary condition is the subtle part. A flat-hold extension past the simulated horizon would be wrong for the carbon price specifically, because `calculate_price_at_time()` returns zero once the policy period ends: the schedule is coded to stop, not to persist. Since `duration_future` typically ends at the policy horizon, a flat hold would have a forward-looking agent treat a temporary tax as permanent at its peak rate. The carbon component is therefore extended using

its own schedule, while base energy prices and the decarbonisation trend are extended by holding their final value, which for those series is the correct continuation rather than an approximation.

This is the channel through which forward-looking agents would anticipate an announced ban or a scheduled carbon price before it arrives, over a horizon implied by the model's own discount and depreciation rates rather than by a separate anticipation parameter.

11.3 Carbon price schedules and the research subsidy

`calculate_growth()` supports `flat`, `linear`, `quadratic` and `exponential` carbon price paths via `Carbon_price_state`. Every configuration here uses `flat`, matching the paper's constant-intensity design, but the other schedules are the hook for time-varying schemes such as the EU ETS. A sixth policy lever, `Research_subsidy`, is present in the policy state dictionary but is not among the five instruments analysed.

12 Implementation notes

Vectorisation. The hot path is the users-by-vehicles utility matrix, rebuilt each step. Utility, masking, exponentiation and pricing are all matrix operations; the only per-agent Python loop is the sequential choice itself, which cannot be vectorised because used car stock must update between choices.

Randomness and reproducibility. Two seeds are separated deliberately. `seed_inputs` governs structural draws (landscapes, network, population attributes), and `seed` governs run-time stochasticity: activation, innovation and production rolls, and choice draws. `handle_seed()` distributes `RandomState` instances to each component. Fixing `seed_inputs` while varying `seed` therefore isolates behavioural noise from structural variation, which is what multi-seed experiments exploit.

Parallelism. Monte Carlo replication is parallel across seeds via `joblib`, with worker count from `get_num_workers()`. Everything is CPU-bound; `torch` is present only because `sbi` requires it for the inference-based calibration, and is pinned to a CPU-only build.

Cost. A single run is 600 months of a 3,000-user, 16-firm market with two NK landscapes. The policy experiments dominate total cost, since each Bayesian optimisation iteration is 64 seeded runs of the future period; controller reuse (Section 7) is what keeps this tractable.

13 Symbol-to-variable map

Symbol	Code	Location
β_i	`beta_vec`, `beta_s_values`	social network, firm segments
γ_i	`gamma_vec`, `gamma_s_values`	social network, firm segments
ν	`nu`	vehicle user parameters
α, ζ	`alpha`, `zeta`	vehicle user parameters
κ	`kappa`	vehicle user parameters
χ_i	`chi_vec`; `a_chi`, `b_chi`	social network
D_i	`d_vec`; `d_mean`	social network; firm
r	`r`	vehicle user parameters
δ	`delta`	ICE/EV parameters
ξ	`delta_P`	ICE/EV parameters

Symbol	Code	Location
η	`prob_switch_car`	social network
ι	`prob_innovate`	firm
θ	`prob_change_production`	firm
λ	`lambda`	firm
μ	`mu`	second-hand market
M^+	`memory_cap`	firm
Y^+	`max_cars_prod`	firm
V^+	`max_num_cars_prop`	second-hand market
T^+	`age_limit_second_hand`	second-hand market
F	`scrap_price`	second-hand market
N, K	`N`, `K`	ICE/EV landscape
E	`production_emissions`	ICE/EV parameters
J, I	`J`, `num_individuals`	firm manager, social network
R^β, R^γ	`num_beta_segments`, `num_gamma_segments`	firm manager
ρ_{WS}, p_{WS}	`SW_network_density`, `SW_prob_rewire`	social network
$Q_{a,t}$, $\omega_{a,t}$, $B_{a,t}$	`Quality_a_t`, `Eff_omega_a_t`, `B`	car attribute dicts
$L_{a,t}$	`L_a_t`	car objects
$z_{v,t}$	`transportType` (2 = ICE, 3 = EV)	car objects
$o_{v,t}$	`owner_id`	car objects
$x_{i,t}$	`consider_ev_vec`	social network
$\psi_{i,t}$	`vehicle_chosen`	`user_chooses()`
$U_{a,i,t}$	`utilities_matrix`	social network
$W_{s,t}$	`W_vec`, `W_segment`	firm manager
$I_{s,t}$	`I_s_t_vec`	firm manager
$P^*_{s,m,t}$	`optimal_price_segments`	car objects
$M_{j,t}$	`list_technology_memory`	firm
$M^{RD}_{j,t}$	`generate_neighbouring_technologies()`	firm
$T_{m,j,t}$	`update_memory_timer()`	firm